App.js

```
import React from 'react';
import { TaskApp } from './TaskApp';

export default function App() {
  return <TaskApp />;
}
```

StudyTips.jsx

```
import React from 'react';

// Default export is mandatory for React.lazy()
export default function StudyTips() {
  return (
    <div className="tips-box">
      <h3>Pro Study Tips Loaded!</h3>
      <p>This component was not downloaded until you clicked the button.</p>
    </div>
  );
}
```

TaskApp.jsx

```
import React, { useState, useMemo, useCallback, lazy, Suspense } from 'react';
import './TaskApp.css';

const StudyTips = lazy(() => import('./StudyTips'));

const TaskItem = React.memo(({ task, onToggle, onDelete }) => {
  console.log(`Drawing Task: ${task.text}`);
```

```
  return (
    <li className="task-item">
      <span className={task.completed ? 'text-done' : 'text-pending'}>
        {task.text}
      </span>
      <div className="task-actions">
        <button className="btn-toggle" onClick={() => onToggle(task.id)}>
          {task.completed ? 'Undo' : 'Complete'}
        </button>
        <button className="btn-delete" onClick={() => onDelete(task.id)}>
          Delete
        </button>
      </div>
    </li>
  );
});

export const TaskApp = () => {
  const [tasks, setTasks] = useState([
    { id: 1, text: 'Learn React Hooks', completed: false },
    { id: 2, text: 'Build a Calculator', completed: true },
  ]);
  const [newTaskText, setNewTaskText] = useState('');

  // UPDATED: Changed from random colors to a clear Dark Mode toggle
  const [isDarkMode, setIsDarkMode] = useState(false);
  const [showTips, setShowTips] = useState(false);

  const completedCount = useMemo(() => {
    console.log('Calculating completed tasks...');
    return tasks.filter(task => task.completed).length;
```

```
}, [tasks]);

const toggleTask = useCallback((id) => {
  setTasks(prevTasks =>
    prevTasks.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  );
}, []);

const deleteTask = useCallback((id) => {
  setTasks(prevTasks => prevTasks.filter(task => task.id !== id));
}, []);

const addTask = (e) => {
  e.preventDefault();
  if (newTaskText.trim() === '') return;

  const newTask = {
    id: Date.now(),
    text: newTaskText,
    completed: false
  };

  setTasks(prevTasks => [...prevTasks, newTask]);
  setNewTaskText('');
};

return (
  // UPDATED: Toggle a specific CSS class based on the state
  <div className={`full-screen-wrapper ${isDarkMode ? 'theme-dark' : 'theme-light'}`}>
    <div className="app-container">
```

```jsx
<header className="header">
  <h2>Smart Task Manager</h2>
  <button onClick={() => setIsDarkMode(!isDarkMode)} className="btn-theme">
    {isDarkMode ? ' Light Mode' : ' Dark Mode'}
  </button>
</header>

<form onSubmit={addTask} className="add-task-form">
  <input
    type="text"
    value={newTaskText}
    onChange={(e) => setNewTaskText(e.target.value)}
    placeholder="Type a new task..."
    className="input-task"
  />
  <button type="submit" className="btn-add">Add</button>
</form>

<div className="stats-box">
  <p>You have completed <strong>{completedCount}</strong> out of {tasks.length} tasks.</p>
</div>

<ul className="task-list">
  {tasks.map(task => (
    <TaskItem
      key={task.id}
      task={task}
      onToggle={toggleTask}
      onDelete={deleteTask}
    />
  ))}
```

```
          {tasks.length === 0 && <p className="empty-msg">No tasks remaining!</p>}
        </ul>

        <div className="footer-actions">
          <button onClick={() => setShowTips(!showTips)} className="btn-tips">
            {showTips ? 'Hide Tips' : 'Load Study Tips'}
          </button>
        </div>

        {showTips && (
          <Suspense fallback={<div className="loading">Downloading tips...</div>}>
            <StudyTips />
          </Suspense>
        )}

      </div>
    </div>
  );
};
```

TaskApp.css

```
/* Full screen wrapper to ensure theme changes are highly visible */
.full-screen-wrapper {
  min-height: 100vh;
  padding: 2rem 1rem;
  transition: background-color 0.4s ease;
  display: flex;
  justify-content: center;
  align-items: flex-start;
}
```

```css
.app-container {
  max-width: 500px;
  width: 100%;
  background-color: white;
  padding: 2rem;
  border-radius: 12px;
  box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.1);
  font-family: system-ui, sans-serif;
  border: 1px solid #e2e8f0;
}

.header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 1.5rem;
}
.header h2 { margin: 0; color: #1e293b; }
.btn-theme {
  background: #cbd5e1;
  color: #0f172a;
  border: none;
  padding: 0.5rem 1rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
}

/* New Form Styles */
.add-task-form {
  display: flex;
  gap: 0.5rem;
```

```
  margin-bottom: 1.5rem;
}
.input-task {
  flex: 1;
  padding: 0.75rem;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 1rem;
}
.btn-add {
  background: #0f172a;
  color: white;
  border: none;
  padding: 0 1.5rem;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
}

.stats-box {
  background: #f8fafc;
  padding: 1rem;
  border-radius: 8px;
  margin-bottom: 1.5rem;
  border-left: 4px solid #3b82f6;
}

.task-list {
  list-style: none;
  padding: 0;
  margin: 0 0 1.5rem 0;
}
```

```css
.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem;
  margin-bottom: 0.5rem;
  border-radius: 8px;
  border: 1px solid #e2e8f0;
}
.text-done { text-decoration: line-through; color: #94a3b8; }
.text-pending { color: #0f172a; font-weight: 500; }

.task-actions {
  display: flex;
  gap: 0.5rem;
}
.btn-toggle {
  background: #3b82f6;
  color: white;
  border: none;
  padding: 0.4rem 0.8rem;
  border-radius: 4px;
  cursor: pointer;
}
.btn-delete {
  background: #ef4444;
  color: white;
  border: none;
  padding: 0.4rem 0.8rem;
  border-radius: 4px;
  cursor: pointer;
}
```

```css
.empty-msg {
  text-align: center;
  color: #94a3b8;
  font-style: italic;
}

.btn-tips {
  background: #10b981;
  color: white;
  border: none;
  padding: 0.75rem 1.5rem;
  border-radius: 6px;
  cursor: pointer;
  width: 100%;
  font-size: 1rem;
}

.tips-box {
  margin-top: 1rem;
  padding: 1rem;
  background: #dcfce7;
  border: 1px solid #86efac;
  border-radius: 8px;
  color: #166534;
}
.loading {
  margin-top: 1rem;
  text-align: center;
  color: #64748b;
}
/* --- DARK MODE OVERRIDES --- */
```

```css
.theme-light {
  background-color: #f8fafc;
}

.theme-dark {
  background-color: #0f172a;
}

.theme-dark .app-container {
  background-color: #1e293b;
  border-color: #334155;
  color: #f8fafc;
}

.theme-dark .header h2 {
  color: #f8fafc;
}

.theme-dark .task-item {
  background-color: #334155;
  border-color: #475569;
}

.theme-dark .text-pending {
  color: #f8fafc;
}

.theme-dark .stats-box {
  background-color: #0f172a;
  border-left-color: #3b82f6;
  color: #cbd5e1;
}
```

```css
.theme-dark .input-task {
  background-color: #0f172a;
  color: white;
  border-color: #475569;
}

.theme-dark .empty-msg {
  color: #cbd5e1;
}
```